

63. Un compte à rebours

Le chronomètre à l'envers : dix secondes, puis « FINI! » — sans jamais passer sous zéro.

1. Ce qu'il faut comprendre

Beaucoup de jeux se jouent **contre la montre** : trouver la sortie avant la fin du temps. C'est le chronomètre de la leçon d'avant, à l'envers : `reste` part de 10 et perd une seconde à chaque paquet de 60 images.

Le piège du zéro (chapitre 3) revient : un octet à 0 qui perd 1 devient **255**. Un compte à rebours sans garde affiche « FINI! », puis repart de 255 secondes. D'où la garde : on ne compte que **si** `reste > 0`.

Ce qui est nouveau ici : un événement, une seule fois. Au moment précis où `reste` tombe à 0, on écrit « FINI! ». Ce `if` est **dans** celui de la seconde écoulée : il ne peut se déclencher qu'une fois, à la dernière seconde, et pas à chaque image pendant tout le reste du jeu.

A relance : `reste` revient à 10, `top` repart de maintenant, et on efface « FINI! ». Le même programme sert donc à plusieurs parties : c'est l'idée qui mènera, au chapitre suivant, aux **états** du jeu.

À toi : fais clignoter le nombre pendant les trois dernières secondes (le modulo du chapitre 1 et un `effacer`).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **910 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	56 o	← la console, à l'allumage — la mise en route de la console
main()	142 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	432 o	← CopierTuiles — 27 lettres sur 43
"COMPTE A REBOURS"	16 o	← texte() — un texte gravé
"A: RELANCER"	11 o	← texte() — un texte gravé
"FINI!"	5 o	← texte() — un texte gravé
" "	5 o	← texte() — un texte gravé

Lettres gravées : ABCEFILMNOPRSTU0123456789!:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 4	44 o sur place, dans main() + EcrireTexte (10 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
images()	× 3	6 o sur place, dans main()

```
bouton() × 1      5 o sur place, dans main() + LireManette (37 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un compte à rebours de 10 secondes ; à 0, FINI!
// A le relance à 10.
//
// Comme le chronomètre, mais à l'envers – avec une GARDE : on ne descend
// que si reste > 0 (sinon, après 0 viendrait 255).

#include <texte>      // écrit un texte à l'écran
#include <nombre>    // écrit un nombre en chiffres
#include <bouton>    // lit un bouton de la manette
```

2. UNE VARIABLE — TOP

```
uint8_t top = 0;          // images() au début de la seconde en cours
```

3. UNE VARIABLE — RESTE

```
uint8_t reste = 10;      // les secondes qui restent
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(3, 3, "COMPTE A REBOURS");
    texte(3, 12, "A: RELANCER");
    nombre(9, 6, reste, 2);                 // 10
    top = images();                          // le départ : maintenant

    while (true) {                           // La boucle du jeu :
        image();

        uint8_t ecole = images() - top;     // les images depuis le début de la seconde
        if (reste > 0 && ecole >= 60) {      // la garde : jamais sous zéro
            top = top + 60;                  // le repère avance d'une seconde
            reste--;                          // une seconde de moins
            nombre(9, 6, reste, 2);
            if (reste == 0) {                 // la dernière seconde, une seule fois
                texte(7, 8, "FINI!");
            }
        }

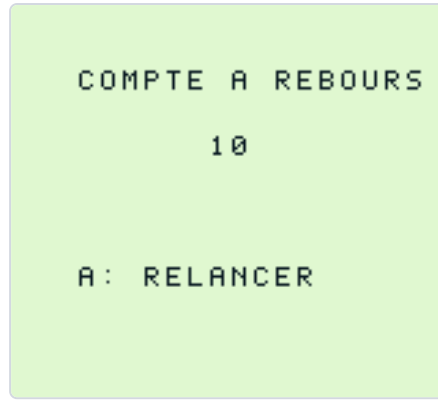
        if (bouton(A)) {                     // A : on relance.
            reste = 10;
            top = images();
            nombre(9, 6, reste, 2);
            texte(7, 8, "    ");             // on efface FINI! (5 espaces pour 5 signes)
        }
    }
}
```

3. Ce qui se passe, étape par étape

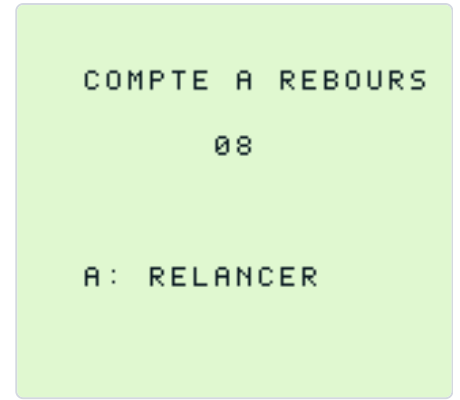
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



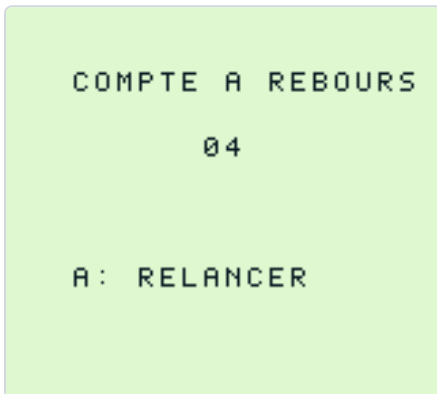
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



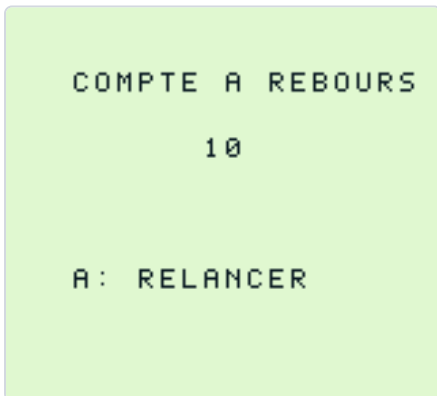
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Un nombre qui descend de 10 à 0, une seconde à la fois, puis « FINI! » ; A relance.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en route, il descend (5)

✓ au bout de dix secondes : 0

✓ FINI! s'affiche

✓ et il ne passe jamais sous zéro

✓ A relance à 10, et efface FINI!

← [62. Compter les secondes : un chronomètre](#)

[sommaire](#)

[64. enum : un nom pour chaque écran](#) →